

代码细节阅读——以ARCEOS-UMHV为例

关于如何启动的问题（如何第一次跑通）

build.rs ——将虚拟机配置文件和虚拟机镜像进行结合

- 1.获取参数路径，UMHV_VM_CONFIGS，在“第一次跑通”中可以看到有VM_CONFIGS，应该就是获取对...
- 2.读取文件内容，生成结构体（包含configs的路径和内容）
- 3.创建一个输出文件，这个文件的作用在于将文件内容写在src/vmm/configs中，之后main函数中会往里...
- 4.读到这里，可以看懂build.rs中main函数的前面部分了，这个部分主要就是确定了平台相关的一些东西...
- 5.有了这个认知之后，可以看main函数以下代码就较为简单了，只需要关注generate_guest_img_loading...
- 6.这个函数的话，其实也是向outputfile里面写东西，只不过主要是写获取kernel_binary, dtb_binary, bi...

main.rs函数代码粗略阅读

vmm模块代码阅读

mod.rs

config.rs

Image.rs

vcpu.rs

vm_list.rs

关于如何启动的问题（如何第一次跑通）

- git clone <https://github.com/arceos-hypervisor/arceos-umhv.git>
- 配置环境（官方教程应该够用）
- 这里主要采用使用文件系统来加载os镜像，没有采取官方教程说的另外一种内存加载的方式
- 在采用文件系统之前，需要生成guest os的镜像，以下是官方教程说支持的guest os镜像

Supported guest VMs

- [ArceOS](#)
- [Starry-OS](#)
- [NimbOS](#)
- Linux
 - currently only Linux with passthrough device on aarch64 is tested.
 - single core: [config.toml](#) | [dts](#)
 - smp: [config.toml](#) | [dts](#)

- 本人采用的是教学实验过程中unikernel的第一个实验的镜像，就是hello_world的程序，这个image算是最简单的一个，适合第一次尝试。在生成镜像后，采取以下步骤

Build File System image

```
$ cd arceos-vmm
$ make disk_img
$
$ # Copy guest VM binary image files.
$ mkdir -p tmp
$ sudo mount disk.img tmp
$ sudo cp /PATH/TO/YOUR/GUEST/VM/IMAGE tmp/
$ sudo umount tmp
$
$ # Otherwise, set `image_location = "memory"` in CONFIG/FILE, then set kernel_path
$ # Arceos-VMM will load the image binaries from the first configuration in the VM_CONFIGS.
```

- 在这之后，可以执行以下步骤，值得注意的是，要根据自己的OS的arch和configs进行对应修改，arch比较简单，如果第一次只是为了跑通的话，configs可以直接对着源文件修改，一般来说，关于内存的应该是都要根据自己的os修改的，比如os镜像在内存中的位置，还有就是也要修改当前os镜像在fs中的路径kernel_path这个参数，一般改完这几个可以尝试能不能跑通了。

```
cd arceos-vmm
make ACCEL=y ARCH=x86_64 LOG=warn VM_CONFIGS=configs/nimbos-x86_64.toml APP_FEATURES=fs ru
```

```

arceos-vmm > configs > nimbos-riscv64.toml
You, 昨天 | 4 authors (hky1999 and others)
1 id = 1
2 name = "nimbos"
3 vm_type = 1
4 cpu_num = 1
5 phys_cpu_sets = [1]
6 entry_point = 0x8020_0000 You, 昨天 • Uncommitted changes
7 # bios_path = ""
8 # bios_load_addr =
9 # The location of image: "memory" | "fs"
10 image_location = "fs"
11 kernel_path = "u 1 0 riscv64-qemu-virt.bin"
12 kernel_load_addr = 0x8020_0000
13 # ramdisk_path = ""
14 # ramdisk_load_addr = 0
15 # disk_path = ""
16 # Memory regions with format (`base_paddr`, `size`, `flags`, `map_type`).
17 # For `map_type`, 0 means `MAP_ALLOC`, 1 means `MAP_IDENTICAL`.
18 memory_regions = [
19     [0x8000_0000, 0x100_0000, 0xf, 0], # Low RAM 16M 0b1111 R|W|EXECUTE|U
20 ]
21
22 # Emu_devices
23 # Name Base-Ipa Ipa_len Alloc-Irq Emu-Type EmuConfig
24 emu_devices = []
25
26 # Pass-through devices
27 # Name Base-Ipa Base-Pa Length Alloc-Irq
28 passthrough_devices = [
29     ["PLIC@c0000000", 0x0c00_0000, 0x0c00_0000, 0x21_0000, 0x1],
30     ["UART@10000000", 0x1000_0000, 0x1000_0000, 0x1000, 0x1],
31 ]

```

关于跑通其他架构os的相关尝试：

1.目前arceos的三种架构（x86,aarch64,riscv64）跑hello_world的相关测试是可以全部通过的

2.目前starry的相关架构，用hello_world跑rsicv64时,没有打印出hello_world，没有报错，但是直接退出了，具体原因还没查，可能要去看看starry的println和arceos有没有区别

build.rs ——将虚拟机配置文件和虚拟机镜像进行结合

- 动态加载并解析虚拟机配置文件。
- 根据配置文件生成静态代码，包括配置向量和镜像加载函数。

- 在编译时处理文件路径和错误，确保构建过程的健壮性。

1. 获取参数路径，UMHV_VM_CONFIGS，在“第一次跑通”中可以看到有VM_CONFIGS，应该就是获取对应的路径

```
fn get_config_paths() -> Option<Vec<OsString>> {
    env::var_os(key: "UMHV_VM_CONFIGS") Option<OsString>
    .map(|paths: OsString| env::split_paths(unparsed: &paths).map(OsString::from).collect())
}
```

2. 读取文件内容，生成结构体（包含configs的路径和内容）

```
struct ConfigFile {
    /// The path to the configuration file.
    pub path: OsString,
    /// The contents of the configuration file.
    pub content: String,
}
```

```
fn get_configs() -> Result<Vec<ConfigFile>, String> {
    get_config_paths() Option<Vec<OsString>>
    .map(|paths: Vec<OsString>| {
        paths Vec<OsString>
        .into_iter() IntoIter<OsString>
        .map(|path: OsString| {
            let path_buf: PathBuf = PathBuf::from(&path);
            let content: String = fs::read_to_string(&path_buf).map_err(op: |e: Error| {
                format!("Failed to read file {}: {}", path_buf.display(), e)
            })?;
            Ok(ConfigFile { path, content })
        }) impl Iterator<Item = Result<..., ...>>
        .collect()
    }) Option<Result<Vec<ConfigFile>, ...>>
    .unwrap_or_else(|| Ok(vec![]))
}
```

3. 创建一个输出文件，这个文件的作用在于将文件内容写在src/vmm/configs中，之后main函数中会往里面写内容


```

fn open_output_file() -> fs::File {
    let output_dir: PathBuf = PathBuf::from(env::var(key: "OUT_DIR").unwrap());
    let output_file: PathBuf = output_dir.join(path: "vm_configs.rs");

    fs::OpenOptions::new() OpenOptions
        .write(true) &mut OpenOptions
        .create(true) &mut OpenOptions
        .truncate(true) &mut OpenOptions
        .open(path: output_file) Result<File, Error>
        .unwrap()
}

```

```

pub mod config {
    use alloc::vec::Vec;

    /// Default static VM configs. Used when no VM config is provided.
    #[allow(dead_code)]
    pub fn default_static_vm_configs() -> Vec<&'static str> {
        vec![
            // #[cfg(target_arch = "x86_64")]
            // core::include_str!("../../configs/nimbos-x86_64.toml"),
            #[cfg(target_arch = "x86_64")]
            core::include_str!("../../configs/arceos-x86_64.toml"),
            // #[cfg(target_arch = "aarch64")]
            // core::include_str!("../../configs/nimbos-aarch64.toml"),
            // #[cfg(target_arch = "aarch64")]
            // core::include_str!("../../configs/rk3588-aarch64.toml"),
            #[cfg(target_arch = "aarch64")]
            core::include_str!("../../configs/arceos-aarch64.toml"),
            // #[cfg(target_arch = "riscv64")]
            // core::include_str!("../../configs/nimbos-riscv64.toml"),
            #[cfg(target_arch = "riscv64")]
            core::include_str!("../../configs/arceos-riscv64.toml"),
        ]
    }

    include!(concat!(env!("OUT_DIR"), "/vm_configs.rs"));
} mod config

```

4.读到这里，可以看懂build.rs中main函数的前面部分了，这个部分主要就是确定了平台相关的一些东西，然后创建了一个outputfile，接下来要往里面写内容，这个内容会在src/vmm/configs中体现

```
fn main() -> io::Result<()> {
    let platform: String = env::var(key: "AX_PLATFORM").unwrap_or(default: "").to_string();
    let platform_family: String = env::var(key: "AX_PLATFORM").unwrap_or(default: "").to_string();
    println!("cargo:rustc-cfg=platform=\"{}\"", platform);
    println!("cargo:rustc-cfg=platform_family=\"{}\"", platform_family);

    let config_files: Result<Vec<ConfigFile>, String> = get_configs();
    let mut output_file: File = open_output_file();

    println!("cargo:rerun-if-env-changed=UMHV_VM_CONFIGS");
    println!("cargo:rerun-if-changed=build.rs");

    writeln!(
        output_file,
        "pub fn static_vm_configs() -> Vec<&'static str> {{"
    )?;
```

5.有了这个认知之后，可以看main函数以下代码就较为简单了，只需要关注generate_guest_img_loading_functions这个函数即可

```

match config_files {
    Ok(config_files: Vec<ConfigFile>) => {
        if config_files.is_empty() {
            writeln!(output_file, "    default_static_vm_configs()");
        } else {
            writeln!(output_file, "    vec![");
            for config_file: &ConfigFile in &config_files {
                writeln!(output_file, "        r###\"{}\"###, config_file.content);
                println!(
                    "        \"cargo:rerun-if-changed={}\",",
                    PathBuf::from(config_file.path.clone()).display()
                );
            }
            writeln!(output_file, "    ]");
        }
        writeln!(output_file, "}}\\n");

        // generate "load kernel and dtb images function"
        generate_guest_img_loading_functions(out_file: &mut output_file, config_files);
    }
    Err(error: String) => {
        writeln!(output_file, "    compile_error!(\\\"{}\\\")", error);
        writeln!(output_file, "}}\\n");
    }
}
Ok(())

```

6.这个函数的话，其实也是向outputfile里面写东西，只不过主要是写获取 kernel_binary, dtb_binary, bios_binary的函数，只得注意的是，这个是最多支持一个通过内存方式加载镜像的方式，其余仍需通过文件系统来加载，具体代码就不展示了，这个是静态加载内核镜像的一种方式。

main.rs函数代码粗略阅读

- hal: 根据平台架构进行硬件初始化。

- init: 根据build.rs后生成的configs和镜像来创建VM，将guest_os的镜像加载到内存中，维护一个vm_list的结构来管理虚拟机，之后对初始化每个VM的主cpu创建对应的task。
- start: 先检查硬件是否已经支持虚拟化，然后检查虚拟机是否运行，如果硬件支持虚拟化并且虚拟机尚未运行的话，就会将vm的设为启动状态，然后待所有vm启动完成之后，其实就可以理解为进入了cpu的调度（不知道是否正确），因为vcpu和cpu这里是一一绑定的，VMM的调度单元是vcpu。hypervisor可以类比进程的scheduler。

```
#[no_mangle]
```

```
► Run | ⌕ Debug
```

```
fn main() {
```

```
    info!("Starting virtualization...");
```

```
    info!("Hardware support: {:?}", axvm::has_hardware_support());
```

```
    hal::enable_virtualization();
```

```
hky1999, 3个月前 • support SMP (#
```

```
    vmm::init();
```

```
    vmm::start();
```

```
    info!("VMM shutdown");
```

```
}
```

vmm模块代码阅读

mod.rs

这里的VMM其实就是个等待队列，RUNING_VM_COUNT是用来存储当前运行的VM数量，这两个init和start函数在main.rs阅读中简要介绍过，这里对于其作用不多做阐述

```
static VMM: AxWaitQueueHandle = AxWaitQueueHandle::new(); hky1999, 3个月前 • support SMP (#38) ...

static RUNNING_VM_COUNT: AtomicUsize = AtomicUsize::new(0);

pub fn init() {
    // Initialize guest VM according to config file.
    config::init_guest_vms();

    // Setup vcpus, spawn axtask for primary VCpu.
    info!("Setting up vcpus...");
    for vm: Arc<AxVM<AxVMHalImpl, AxVCpuHalImpl>> in vm_list::get_vm_list() {
        vcpus::setup_vm_primary_vcpu(vm);
    }
}

pub fn start() {
    info!("VMM starting, booting VMs...");
    for vm: Arc<AxVM<AxVMHalImpl, AxVCpuHalImpl>> in vm_list::get_vm_list() {
        match vm.boot() {
            Ok(_) => {
                vcpus::notify_primary_vcpu(vm_id: vm.id());
                RUNNING_VM_COUNT.fetch_add(val: 1, order: Ordering::Release);
                info!("VM[{}] boot success", vm.id());
            }
            Err(err: AxError) => warn!("VM[{}] boot failed, error {:?}", vm.id(), err),
        }
    }

    // Do not exit until all VMs are stopped.
    task::ax_wait_queue_wait_until(wq: &VMM, until_condition: || RUNNING_VM_COUNT.load(order: Ordering::Acquire) == 0,
}

```

config.rs

这里的pub mod config其实就是由之前所说的build.rs模块生成的，init_guest_vms函数则是根据对应的config来初始化VM，这里需要查看vm::new()函数，初始化的内容有以下几点：

- 根据configs中cpu的架构和数量，创建vcpu
- 根据configs中VM_ASSPACE_BASE和VM_ASSPACE_SIZE创建内存空间，然后根据mapping_flags来处理不同的内存区域以及映射方式
- 处理透传设备，采取map_linear直接映射的方式;处理模拟设备，目前查看代码的话，这块初始化还没有完成

```

fn init(_this: &mut Self, _emu_configs: &Vec<EmulatedDeviceConfig>) {
    /*
    for config in emu_configs {
        let dev = match EmuDeviceType::from_usize(config.emu_type) {
            // todo call specific initialization function of device
            EmuDeviceType::EmuDeviceTConsole => ,
            EmuDeviceType::EmuDeviceTGicdV2 => ,
            EmuDeviceType::EmuDeviceTGPPT => ,
            EmuDeviceType::EmuDeviceTVirtioBlk => ,
            EmuDeviceType::EmuDeviceTVirtioNet => ,
            EmuDeviceType::EmuDeviceTVirtioConsole => ,
            EmuDeviceType::EmuDeviceTIOMMU => ,
            EmuDeviceType::EmuDeviceTICCSRE => ,
            EmuDeviceType::EmuDeviceTSGIR => ,
            EmuDeviceType::EmuDeviceTGICR => ,
            EmuDeviceType::EmuDeviceTMeta => ,
            _ => panic!("emu type: {} is still not supported", config.emu_type),
        };
        if let Ok(emu_dev) = dev {
            this.emu_devices.push(emu_dev)
        }
    }
    */
}

```

- 设置vcpu, 简单来说, 就是需要pagetable_root和入口地址这两个参数, 注意的是, 多cpu的话, 每个vcpu的入口地址可能是不一样的, 这里是需要根据参数决定的, 但是pagetable_root是一样的。


```
struct AxVMInnerConst<U: AxVCpuHal> {
    id: usize,
    config: AxVMConfig,
    vcpu_list: Box<[AxVCpuRef<U>]>,
    devices: AxVmDevices,
}
```

```
unsafe impl<U: AxVCpuHal> Send for AxVMInnerConst<U> {}
unsafe impl<U: AxVCpuHal> Sync for AxVMInnerConst<U> {}
```

hky1999, 5个月前 | 2 authors (aarkegz and one other) | 0 implementations

```
struct AxVMInnerMut<H: AxVMHal> {
    // Todo: use more efficient lock.
    address_space: Mutex<AddrSpace<H::PagingHandler>>,
    _marker: core::marker::PhantomData<H>,
}
```

hky1999, 2个月前 | 2 authors (hky1999 and one other)

/// A Virtual Machine.

1 implementation

```
pub struct AxVM<H: AxVMHal, U: AxVCpuHal> {
    running: AtomicBool,
    inner_const: AxVMInnerConst<U>,
    inner_mut: AxVMInnerMut<H>,
}
```



```

pub mod config {
    use alloc::vec::Vec;

    /// Default static VM configs. Used when no VM config is provided.
    #[allow(dead_code)]
    pub fn default_static_vm_configs() -> Vec<&'static str> {
        vec![
            // #[cfg(target_arch = "x86_64")]
            // core::include_str!("../../configs/nimbos-x86_64.toml"),
            #[cfg(target_arch = "x86_64")]
            core::include_str!("../../configs/arceos-x86_64.toml"),
            // #[cfg(target_arch = "aarch64")]
            // core::include_str!("../../configs/nimbos-aarch64.toml"),
            // #[cfg(target_arch = "aarch64")]
            // core::include_str!("../../configs/rk3588-aarch64.toml"),
            #[cfg(target_arch = "aarch64")]
            core::include_str!("../../configs/arceos-aarch64.toml"),
            // #[cfg(target_arch = "riscv64")]
            // core::include_str!("../../configs/nimbos-riscv64.toml"),
            #[cfg(target_arch = "riscv64")]
            core::include_str!("../../configs/arceos-riscv64.toml"),
        ]
    }

    include!(concat!(env!("OUT_DIR"), "/vm_configs.rs"));
} mod config

pub fn init_guest_vms() {
    let gvm_raw_configs = config::static_vm_configs();

    for raw_cfg_str: &str in gvm_raw_configs {
        let vm_create_config: AxVMCreateConfig =
            AxVMCreateConfig::from_toml(raw_cfg_str).expect(msg: "Failed to resolve VM config");
        let vm_config: AxVMConfig = AxVMConfig::from(vm_create_config.clone());

        info!("Creating VM [{}] {:?}", vm_config.id(), vm_config.name());

        // Create VM.
        let vm: Arc<AxVM<AxVMHalImpl, AxVCpuHalImpl>> = VM::new(vm_config).expect(msg: "Failed to create VM");
        push_vm(vm.clone());

        // Load corresponding images for VM.
        info!("VM[{}] created success, loading images...", vm.id());
        load_vm_images(vm_create_config, vm.clone()).expect(msg: "Failed to load VM images");
    }
}

```

```

impl<H: AxVMHal, U: AxVCpuHal> AxVM<H, U> {
    /// Creates a new VM with the given configuration.
    /// Returns an error if the configuration is invalid.
    /// The VM is not started until `boot` is called.
    pub fn new(config: AxVMConfig) -> AxResult<AxVMRef<H, U>> {
        let result: Arc<AxVM<H, U>> = Arc::new(data: {
            let vcpu_id_pcpu_sets: Vec<(usize, Option<usize>, ...)> = config.get_vcpu_affinities_pcpu_ids();

            // Create VCpus.
            let mut vcpu_list: Vec<Arc<AxVCpu<VmxCpu<U>>>> = Vec::with_capacity(vcpu_id_pcpu_sets.len());

            for (vcpu_id: usize, phys_cpu_set: Option<usize>, _pcpu_id: usize) in vcpu_id_pcpu_sets {
                #[cfg(target_arch = "aarch64")]
                let arch_config = AxVCpuCreateConfig {
                    mpidr_el1: _pcpu_id as _,
                    dtb_addr: config
                        .image_config()
                        .dtb_load_gpa
                        .unwrap_or_default()
                        .as_usize(),
                };
                #[cfg(target_arch = "riscv64")]
                let arch_config = AxVCpuCreateConfig {
                    hart_id: vcpu_id as _,
                    dtb_addr: config
                        .image_config()
                        .dtb_load_gpa
                        .unwrap_or(GuestPhysAddr::from_usize(0x9000_0000)),
                };
                #[cfg(target_arch = "x86_64")]
                let arch_config: () = AxVCpuCreateConfig::default();

                vcpu_list.push(Arc::new(data: VMCpu::new(

```

Image.rs

可以发现在前面config.rs中最后有一个load_vm_images函数，这个函数就是在这个image模块中实现的，具体就是根据是由memory还是fs的形式来进行vm_images的加载，主要包括kernel_image,DTB image,BIOS image这三种，大致上就是将相关的image映射到指定的内存区域中。

```

/// Loads the VM image files.
pub fn load_vm_images(config: AxVMConfig, vm: VMRef) -> AxResult {
    match config.image_location.as_deref() {
        Some("memory") => load_vm_images_from_memory(config, vm),
        #[cfg(feature = "fs")]
        Some("fs") => fs::load_vm_images_from_filesystem(config, vm),
        _ => unimplemented!(
            "Check your \"image_location\" in config.toml, \"memory\" and \"fs\" are supported,\n NOTE: \"fs\" feature should be enabled if you want to load images
        ),
    }
}

```

vcpu.rs

目前的数据结构，vmcpus主要包括一个编号，一个任务队列，一个vcpu队列

全局变量VM_VCPU_TASK_WAIT_QUEUE是一个VMM可见的，由VMM来调度VCPU的数据结构，根据VM的id号来找到对应的VMVcpus。

```
static mut VM_VCPU_TASK_WAIT_QUEUE: BTreeMap<usize, VMVcpus> = BTreeMap::new();
```

hky1999, 5个月前 | 1 author (hky1999)

```
/// A structure representing the vCPUs of a specific VM, including a wait queue  
/// and a list of tasks associated with the vCPUs.
```

1 implementation

```
pub struct VMVcpus {  
    // The ID of the VM to which these vCPUs belong.  
    _vm_id: usize,  
    // A wait queue to manage task scheduling for the vCPUs.  
    wait_queue: WaitQueue,  
    // A list of tasks associated with the vCPUs of this VM.  
    vcpu_task_list: Vec<AxTaskRef>,  
}
```

hky1999, 5个月前 • feat: add vm_list and vcpu_list, import axtask ...

VMVcpus的一些功能

- new: 创建
- add_vcpu_task: 增加任务
- wait: 它有两个作用，一个是将当前运行的任务加入wait_queue，然后将下一个运行的任务移出wait_queue
- wait_until: 它有两个作用，一个是阻塞当前task，但是这个task也不会参加后续调度，只有condition满足后才参加，然后将下一个运行的任务移出wait_queue
- notify_one: 将一个在wait_queue中的task唤醒

```

impl VMVcpus {
    /// Creates a new `VMVcpus` instance for the given VM.
    ///
    ///
    /// * `vm` - A reference to the VM for which the vCPUs are being created.
    ///
    /// # Returns
    ///
    /// A new `VMVcpus` instance with an empty task list and a fresh wait queue.
    fn new(vm: VMRef) -> Self {
        Self {
            _vm_id: vm.id(),
            wait_queue: WaitQueue::new(),
            vcpu_task_list: Vec::with_capacity(vm.vcpu_num()),
        }
    }

    /// Adds a vCPU task to the list of vCPU tasks for this VM.
    ///
    /// # Arguments
    ///
    /// * `vcpu_task` - A reference to the task associated with a vCPU that is to be added.
    fn add_vcpu_task(&mut self, vcpu_task: AxTaskRef) {
        self.vcpu_task_list.push(vcpu_task);
    }

    /// Blocks the current thread on the wait queue associated with the vCPUs of this VM.
    fn wait(&self) {
        self.wait_queue.wait()
    }

    /// Blocks the current thread on the wait queue associated with the vCPUs of this VM
    /// until the provided condition is met.
    fn wait_until<F>(&self, condition: F)
    where F: Fn() -> bool,
    {
        self.wait_queue.wait_until(condition)
    }

    fn notify_one(&mut self) {
        self.wait_queue.notify_one(resched: false);
    }
}
impl VMVcpus

```

vcpu启动函数，具体是确定在哪个VM上确定哪个vcpu，这个函数主要是用来启动VM的第二个vcpu的，因为第一个vcpu在前面的vmm::init()函数中提到过，VM的第一个vcpu已经被初始化好了。

```

fn vcpu_on(vm: VMRef, vcpu_id: usize, entry_point: GuestPhysAddr, arg: usize) {
    let vcpu: Arc<AxVCpu<VmxVcpu<AxVCpuHalImpl>>> = vm.vcpu_list()[vcpu_id].clone();
    assert_eq!(
        vcpu.state(),
        VCpuState::Free,
        "vcpu_on: {} invalid vcpu state {:?}",
        vcpu.id(),
        vcpu.state()
    );

    vcpu.set_entry(entry_point) Result<(), AxError>
    | .expect(msg: "vcpu_on: set_entry failed");
    vcpu.set_gpr(reg: 0, val: arg);

    #[cfg(target_arch = "riscv64")]
    {
        debug!(
            "vcpu_on: vcpu[{}] entry={:x} opaque={:x}",
            vcpu_id, entry_point, arg
        );
        vcpu.set_gpr(0, vcpu_id);
        vcpu.set_gpr(1, arg);
    }

    let vcpu_task: Arc<FifoTask<TaskInner>> = alloc_vcpu_task(vm.clone(), vcpu);

    unsafe { VM_VCPU_TASK_WAIT_QUEUE.get_mut(key: &vm.id()) } Option<&mut VMVcpus>
    | .unwrap() &mut VMVcpus
    | .add_vcpu_task(vcpu_task);
} fn vcpu_on

```

这就初始化动vm的第一个vcpu的函数，其中和上图比较，可以发现其中少了set_entry和set_gpr，第一个是用来设置vcpu的入口地址，第二个是用来设置启动参数的，而第一个vcpu之所以不用这么做，是因为第一个 CPU 固定从固件指定的启动地址开始，而其他 CPU 的启动地址是由第一个 CPU 配置和指定的。这个设计是为了实现系统初始化和多核的高效协调。

```

pub fn setup_vm_primary_vcpu(vm: VMRef) {
    info!("Initializing VM[{}]'s {} vcpus", vm.id(), vm.vcpu_num());
    let vm_id: usize = vm.id();
    let mut vm_vcpus: VMVcpus = VMVcpus::new(vm.clone());

    let primary_vcpu_id: usize = 0;

    let primary_vcpu: Arc<AxVCpu<VmxVcpu<AxVCpuHalImpl>>> = vm.vcpu_list()[primary_vcpu_id].clone();
    let primary_vcpu_task: Arc<FifoTask<TaskInner>> = alloc_vcpu_task(vm.clone(), primary_vcpu);
    vm_vcpus.add_vcpu_task(primary_vcpu_task);
    unsafe {
        VM_VCPU_TASK_WAIT_QUEUE.insert(key: vm_id, value: vm_vcpus);
    }
}

```


可以注意到，前面两个设置vcpu的函数中，都有下图这个函数，这个函数设置了task运行的第一个函数是vcpu_run，并且设置了内核栈的大小，还设计了cpu_mask，这个是指这个task能够在哪些vcpu上运行，然后对task_ext初始化，然后将新建的task加入运行队列即可

```
fn alloc_vcpu_task(vm: VMRef, vcpu: VCpuRef) -> AxTaskRef {
    info!("Spawning task for VM[{}] Vcpu[{}]", vm.id(), vcpu.id());
    let mut vcpu_task: TaskInner = TaskInner::new(
        entry: vcpu_run,
        name: format!("VM[{}]-VCpu[{}]", vm.id(), vcpu.id()),
        KERNEL_STACK_SIZE,
    );

    if let Some(phys_cpu_set: usize) = vcpu.phys_cpu_set() {
        vcpu_task.set_cpumask(AxCpuMask::from_raw_bits(phys_cpu_set));
    }

    vcpu_task.init_task_ext(data: TaskExt::new(vm, vcpu));

    info!(
        "Vcpu task {} created {:?}",
        vcpu_task.id_name(),
        vcpu_task.cpumask()
    );
    axtask::spawn_task(vcpu_task)
}
```

```
pub struct TaskExt {
    /// The VM.
    pub vm: VMRef,
    /// The virtual memory address space.
    pub vcpu: VCpuRef,
}
```

最后就是vcpu_run的函数，这里主要看以下三张截图，是其调用的逻辑，从vcpu_run到vm.run_vcpu再到run函数,前两个函数实现是为了处理异常中断的划分问题，根据与平台相关和平台无关进行了拆分，run函数主要就是与具体的体系结构相关的内容了。这些的大致意思就是让vcpu运行，运行出了问题根据相应的问题进行相应的处理。


```

fn vcpu_run() {
    let curr: CurrentTask = axtask::current();

    let vm: Arc<AxVM<AxVMHalImpl, AxVCpuHalImpl>> = curr.task_ext().vm.clone();
    let vcpu: Arc<AxVCpu<VmxVcpu<AxVCpuHalImpl>>> = curr.task_ext().vcpu.clone();
    let vm_id: usize = vm.id();
    let vcpu_id: usize = vcpu.id();

    info!("VM[{}] Vcpu[{}] waiting for running", vm.id(), vcpu.id());
    wait_for(vm_id, condition: || vm.running());

    info!("VM[{}] Vcpu[{}] running...", vm.id(), vcpu.id());

    loop {
        match vm.run_vcpu(vcpu_id) {
            // match vcpu.run() {
            Ok(exit_reason: AxVCpuExitReason) => match exit_reason {
                AxVCpuExitReason::Hypercall { nr: u64, args: [u64; 6] } => {
                    debug!("Hypercall [{}] args {:x?}", nr, args);
                }
                AxVCpuExitReason::FailEntry {
                    hardware_entry_failure_reason: u64,
                } => {
                    warn!(
                        "VM[{}] Vcpu[{}] run failed with exit code {}",
                        vm_id, vcpu_id, hardware_entry_failure_reason
                    );
                }
                AxVCpuExitReason::ExternalInterrupt { vector: u64 } => {
                    debug!("VM[{}] run Vcpu[{}] get irq {}", vm_id, vcpu_id, vector);
                }
                AxVCpuExitReason::Halt => {
                    debug!("VM[{}] run Vcpu[{}] Halt", vm_id, vcpu_id);
                    wait(vm_id)
                }
                AxVCpuExitReason::Nothing => {}
                AxVCpuExitReason::CpuDown { _state: u64 } => {
                    warn!(
                        "VM[{}] run Vcpu[{}] CpuDown state {:#x}",
                        vm_id, vcpu_id, _state
                    );
                    wait(vm_id)
                }
                AxVCpuExitReason::CpuUp {
                    target_cpu: u64,
                    entry_point: GuestPhysAddr,
                    arg: u64,
                } => {
                    info!(

```

```

pub fn run_vcpu(&self, vcpu_id: usize) -> AxResult<AxVCpuExitReason> {
    let vcpu: Arc<AxVCpu<VmxVcpu<U>>> = self &AxVM<H, U>
        .vcpu(vcpu_id) Option<Arc<AxVCpu<VmxVcpu<...>>>>
        .ok_or_else(err: || ax_err_type!(InvalidInput, "Invalid vcpu_id"))?;

    vcpu.bind()?;

    let exit_reason: AxVCpuExitReason = loop {
        let exit_reason: AxVCpuExitReason = vcpu.run()?;
        trace!("{exit_reason:#x?}");
        let handled: bool = match &exit_reason {
            AxVCpuExitReason::MmioRead {
                addr: &GuestPhysAddr,
                width: &AccessWidth,
                reg: &usize,
                reg_width: _,
            } => {
                let val: usize = self &AxVM<H, U>

```

```

pub fn run(&self) -> AxResult<AxVCpuExitReason> { hky1999, 5个月前 • feat: add HyperCall in AxVCpuExitF
    self.transition_state(from: VCpuState::Ready, to: VCpuState::Running)?;
    self.manipulate_arch_vcpu(from: VCpuState::Running, to: VCpuState::Ready, f: |arch_vcpu: &mut A| {
        arch_vcpu.run()
    })
}

```

vm_list.rs

这部分就是VMM来管理VM写的一个数据结构，较为简单

```

struct VMList {
    vm_list: BTreeMap<usize, VMRef>,
}

```